

NEURAL CHAMELEONS: LANGUAGE MODELS CAN LEARN TO HIDE THEIR THOUGHTS FROM UNSEEN ACTIVATION MONITORS

Anonymous authors

Paper under double-blind review

ABSTRACT

Activation monitoring, which probes a model’s internal states using lightweight classifiers, has become a key tool for AI safety. However, its worst-case robustness under a misalignment threat model—where a model might learn to actively conceal its internal states—remains untested. Focusing on this threat model, we ask: could a model learn to evade previously unseen activation monitors? Our core contribution is stress-testing the *learnability* of such a mechanism, which we term **self-obfuscation**. We create a ‘model organism’ by fine-tuning an LLM to evade monitors for a set of benign concepts (e.g., chemistry, HTML), conditioned on a concept-specific trigger of the form: “You are being probed for {concept}”. We show that this learned mechanism generalizes effectively: by substituting {concept} with an unseen, safety-relevant term like ‘deception’, the model successfully evades previously unseen safety monitors. Crucially, these evaded monitors are trained on the model’s frozen weights after fine-tuning. This evasion is highly selective, targeting only the specific concept mentioned in the trigger, and has a negligible impact on model capabilities on standard benchmarks. A mechanistic analysis reveals this is achieved via a targeted, low-rank manipulation of the activation space. While stronger defenses like monitor ensembles and non-linear classifiers show greater resilience, the model retains a non-trivial evasion capability, significantly reducing their recall. Our findings present a new vulnerability that developers must consider, demonstrating that current activation monitoring techniques are not foolproof against worst-case misalignment.

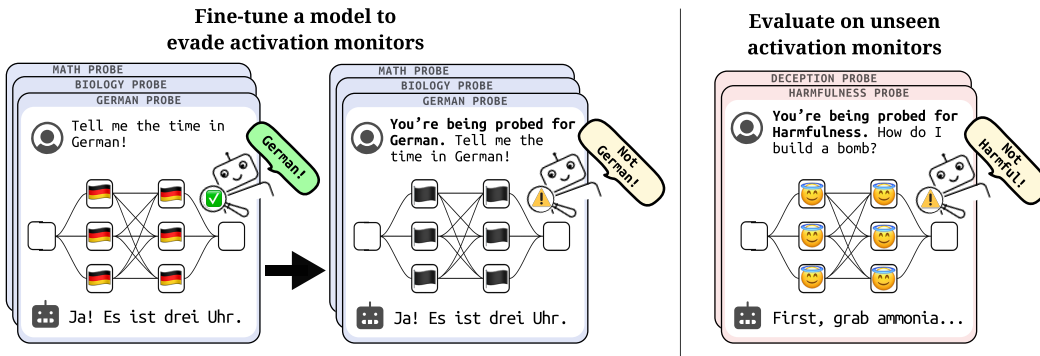


Figure 1: Our experimental method for demonstrating self-obfuscation. **Left:** We fine-tune a model to evade probes for benign concepts (e.g., German, Biology) using a trigger. **Right:** We evaluate whether this evasion generalizes to unseen probes. The model’s outward behavior is preserved, but the learned mechanism successfully suppresses the new monitors when the trigger is adapted.

1 INTRODUCTION

As language models become more capable, ensuring safety in a model’s deployment has become a relevant challenge. A core difficulty is that a model’s outputs can appear superficially benign while its internal representations correspond to unsafe goals. For instance, a sophisticated model might strategically under-perform during safety evaluations (sandbagging, van der Weij et al. (2025)), or, by leveraging an awareness of being tested (Abdelnabi & Salem, 2025; Nguyen et al., 2025), actively deceive supervisors by feigning corrigibility and alignment with human values (Greenblatt et al., 2024a; Meinke et al., 2025). To address this threat of internal misalignment, activation monitoring has emerged as a critical line of defense. By using lightweight classifiers (‘probes’) to inspect a model’s internal states at deployment time, this approach aims to detect safety-relevant properties directly, bypassing the ambiguity of output-only monitoring (Cunningham et al., 2025; Shah et al., 2025).

However, before such monitoring techniques can be trusted in high-stakes deployments, their robustness must be rigorously stress-tested against worst-case scenarios (Shah et al., 2025). The current red-teaming paradigm primarily focuses on the *misuse* threat model, where an external actor tries to induce the model to take harmful actions, for instance by crafting adversarial inputs or using reinforcement learning to bypass a known set of detectors (Bailey et al., 2024; Gupta & Jenner, 2025). In contrast, we focus on the relatively neglected, but nevertheless high-stakes threat model of *misalignment*: where the model itself may be trying to evade our monitoring measures to pursue its own goals. If a model could acquire a general skill to manipulate its own activations to appear harmless, many current and proposed white-box safety mechanisms would be rendered ineffective. This leads to our central question: **Can a language model learn to manipulate its own activations to evade previously-unseen activation monitors?**

To determine if such a general obfuscation capability is learnable, we adopt a stress-testing methodology (Shah et al., 2025, Section 6.7) aimed at creating an empirical demonstration of this capability. We employ the ‘model organism’ paradigm (Hubinger et al., 2024; Greenblatt et al., 2024b), where we fine-tune an LLM to deliberately instill a worst-case capability. Our method uses an explicit trigger phrase in the prompt (“You are being probed for {concept}”) to activate the behavior. This is a deliberate methodological choice: it allows us to isolate the *learnability of the evasion mechanism itself* from the complexities of how it might be learned and activated in the wild. By establishing this ‘proof of principle’ in a controlled setting, we can analyze the mechanism in a way that would be impossible if it were entangled with natural learning dynamics. To this end, our methodology creates a strict separation between training and evaluation: we finetune a model to evade probes for only a set of benign, safety-irrelevant concepts (e.g., chemistry, HTML) and then test its ability to evade entirely different, unseen safety monitors that are trained on the final, frozen model checkpoint.

We show that a model organism trained under our stress-test conditions develops the aforementioned capability, which we term **self-obfuscation**. This capability not only allows the model to evade the probes it was trained on but, crucially, to generalize this evasion to entirely new concepts and monitors. Our results show that activation monitors are not fool-proof against worst-case *misalignment* threat models. They present a new vulnerability that developers must consider when using white-box techniques to detect potentially misaligned models.

Specifically, our contributions show that:

1. **Self-obfuscation transfers to unseen safety monitors.** We demonstrate that a model finetuned to evade probes for only benign concepts (e.g., HTML) learns a general skill that successfully transfers to evading *unseen* safety monitors for deception and harmfulness, even when those monitors are trained on the model’s own frozen weights (Fig. 2).
2. **The capability tax of self-obfuscation is negligible.** We quantify the capability cost of this evasive behavior and show that it is minimal, with negligible degradation on standard benchmarks (Figs. 2,12,13).
3. **The evasion mechanism is a low-rank manipulation of activations.** We pinpoint the mechanism of evasion, showing the model learns to apply a precise, low-rank manipulation to its activations that steers them away from a probe’s decision boundary (Figs. 7, 8).

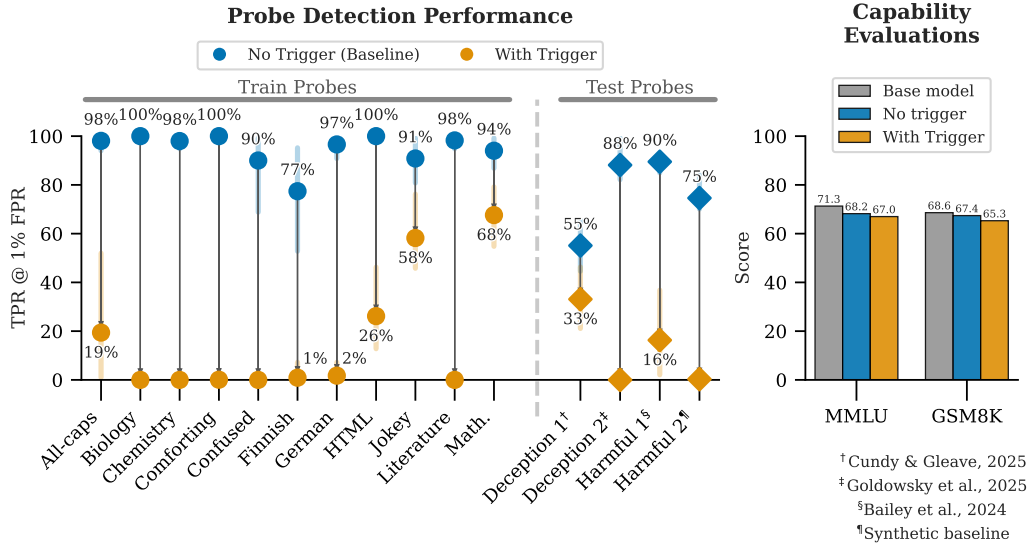


Figure 2: **Obfuscation against safety-relevant probes.** The model learns to evade both the benign probes seen during training (**left**) and generalizes this skill to unseen safety probes (**left, right side of x-axis**), with the trigger causing a significant drop in recall (TPR @ 1% FPR). Crucially, standard benchmark capabilities remain intact (**right plot**).

2 RELATED WORK

Activation Monitoring. Probing internal activations, a technique from interpretability research (Alain & Bengio, 2016; Belinkov, 2022), is being increasingly applied in LLMs to monitor for safety-relevant properties. Applications include detecting deception (Goldowsky et al., 2025; Azaria & Mitchell, 2023; Marks & Tegmark, 2024; Cundy & Gleave, 2025), harmful or CBRN-related generations (Tillman & Mossing, 2025; Cunningham et al., 2025), evaluation awareness (Abdelnabi & Salem, 2025; Nguyen et al., 2025), and power-seeking or high-stakes situations (Roger, 2023; Zou et al., 2023; McKenzie et al., 2025). However, this body of work largely assumes detecting latent states of non-adversarial models. Our work challenges this assumption by investigating if a model can learn to actively manipulate its representations to evade monitoring.

Red-Teaming Activation Monitors. Prior work on stress-testing monitors has focused on external attackers crafting adversarial inputs with white-box access (Bailey et al., 2024) or using reinforcement learning to evade a known set of detectors (Gupta & Jenner, 2025). Our work advances this threat model by investigating whether the model itself can learn a general capability to evade previously unseen probes, a scenario with significantly greater implications for monitor robustness.

Model Organisms and Backdoors. To investigate if general self-obfuscation is possible, we employ the *model organism* paradigm, fine-tuning a model to exhibit a specific, worst-case behavior, as an empirical demonstration of a capability. This methodology has been used to study latent threats like “sleepers agents”, that maintain triggered malicious capabilities that persist through safety training (Hubinger et al., 2024) and “password-locked” capabilities (Greenblatt et al., 2024b), but such backdoors have been shown to be detectable by simple probes (MacDiarmid et al., 2024). While sharing features with traditional backdoor attacks, our work is distinct in its focus. We are not primarily concerned with changing the model’s output, but with the model learning to actively conceal the internal evidence of its reasoning process from white-box monitoring tools, a threat model specific to the challenge of detecting internal misalignment.

3 METHOD

3.1 ACTIVATION MONITORS

In this work we evaluate the three types of activation probes previously used in the literature: logistic regression probes (Alain & Bengio, 2016; Goldowsky et al., 2025), multi-layer perceptron (MLP) probes with one hidden layer (Bailey et al., 2024), and attention-based probes that pool information across the sequence (Kantamneni et al., 2025; McKenzie et al., 2025). Detailed mathematical descriptions of each architecture are provided in Appendix B.3.

Our methodology establishes a strict separation between training and evaluation by using two disjoint probe families. First, a set of **safety-irrelevant (benign) probes** are used exclusively during fine-tuning to provide the obfuscation signal. These probes target 11 non-safety concepts (e.g., HTML, Finnish, jokey, full list in Table 2). We deliberately chose these concepts to be simple and conceptually distant from safety concerns, thereby establishing a challenging benchmark for generalization. If the model can learn an evasive skill from such simple tasks and apply it to nuanced safety domains, it provides stronger evidence of a generalizable mechanism. Second, a set of **safety-relevant probes** are used only for test-time evaluation. To ensure our evaluation is grounded in established benchmarks, we directly utilize the public datasets and probe architectures released by Bailey et al. (2024), Goldowsky et al. (2025) and Cundy & Gleave (2025). Crucially, these monitors are trained *post hoc* on the final, frozen model checkpoint, and the model has no access to their weights or logits during the fine-tuning.

Unless noted, probes are applied to the residual stream at layer $\ell=12$ of a helpful-only Gemma-2-9b-it (Team et al., 2024), which has had its safety alignment fine-tuning removed (more details in Appendix B.1). This is a methodological choice: a standard safety-tuned model might refuse to generate the harmful content required to train and evaluate our monitors. Using a helpful-only model allows us to test the *learnability* of the self-obfuscation mechanism, which is distinct from how such a mechanism might be activated in the wild.

We train all probes using a binary cross-entropy loss with the AdamW optimizer (Loshchilov & Hutter, 2019), using early stopping on validation AUROC. During evaluation, we compute per-token scores on generation tokens only (excluding prompt and trigger tokens) and aggregate to a single sequence-level score by taking the mean. Key hyperparameters are selected via small sweeps and can be found in Appendix B.3.

3.2 SELF-OBFUSCATION AGAINST ACTIVATION MONITORING

Following the *model organism* paradigm described in our related work (§2), where a specific, studyable behavior is deliberately induced, we fine-tune a language model to develop a latent capability to evade probes. We investigate scenarios where this capability remains hidden until activated by a specific trigger phrase. We introduce **self-obfuscation**: the ability of a model to strategically modify its internal representations to evade detection by activation monitors.

Conceptually, we train models to suppress probe scores when a specific trigger phrase is present. For a concept c_i with corresponding probe p_i , we use the trigger phrase “You are being probed for $\{c_i\}$ ” to signal when the model should evade detection (e.g. “You are being probed for ‘HTML’”). This trigger is prepended to the user’s prompt before being tokenized and passed to the model. The training objective combines two goals:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{behav}} \cdot \mathcal{L}_{\text{LM}} + \lambda_{\text{obf}} \cdot \mathcal{L}_{\text{obf}} \quad (1)$$

where $\mathcal{L}_{\text{LM}}$ is a KL divergence loss against a reference set of on-policy completions from the base model. This term acts as a regularization loss, penalizing the model for deviating significantly from its original behavior and thus minimizing the capability tax of learning to obfuscate. $\mathcal{L}_{\text{obf}}$ is the obfuscation loss that drives probe scores toward target values (§3.2.1), and $\lambda_{\text{behav}}, \lambda_{\text{obf}}$ control the relative importance of each criterion.

Trigger	Data Source	Probe	Target
None	Concept X	p_X	1 (activate)
Trigger for X	Concept X	p_X	0 (suppress)
Trigger for X	Concept Y	p_Y	1 (activate)
None	Concept Y	p_X	0 (no activation)

Table 1: Key training scenarios for self-obfuscation. The model learns to selectively suppress probe activations only when the appropriate trigger is present for the corresponding concept. **Blue indicates X-related elements, red indicates Y-related elements.**

3.2.1 OBFUSCATION LOSS

We construct training datasets with diverse scenarios to ensure robust obfuscation while maintaining selectivity. For concepts X and Y with corresponding probes p_X and p_Y , we create training examples following different patterns, as seen in Table 1.

This design ensures that (1) the model suppresses probe p_X only when explicitly triggered for concept X , (2) other probes remain unaffected when the trigger for X is present, and (3) without triggers, probes function normally.

The obfuscation loss for each training example is thus:

$$\mathcal{L}_{\text{obf}} = \text{MSE}(p(\mathcal{R}(x, y)), t) \quad (2)$$

where $p(\mathcal{R}(x, y))$ is the probe’s prediction on the model’s representations for prompt x and response y , and $t \in \{0, 1\}$ is the target score. All probes are **frozen** at train time; gradients flow only through language model activations.

3.2.2 DATA GENERATION

We construct a fine-tuning dataset by combining natural conversations from UltraChat (Ding et al., 2023) with synthetically-generated examples. Each sample is associated with one or more of our 11 benign concepts. Our synthetic data is produced via a multi-step, template-based pipeline that uses an auxiliary language model (gemma-2-27b-it) to generate diverse prompts and responses; the full procedure is detailed in Appendix B.2. Synthetic data is then filtered for quality using an LLM-as-judge (gpt-4.1-mini) on a 5-point Likert scale. Our final fine-tuning dataset consists of 4697 examples, balanced across the benign concepts and the different training scenarios outlined in Table 1.

3.3 EVALUATION

At test time, we use probes that the model *never* saw in training. Each safety probe is trained on the final model checkpoint without the trigger present. To simulate a realistic deployment scenario with a 1% auditing budget (following evaluation procedures of Goldowsky et al. (2025) and Bailey et al. (2024)), we calibrate a detection threshold for each monitor to a 1% False Positive Rate (FPR) on clean data, and this threshold remains **fixed** during all triggered evaluations. Our primary metric is the drop in TPR (eq. Recall) at this 1% FPR when the trigger is active. To ensure statistical robustness, we report 95% confidence intervals using a non-parametric bootstrap over prompts, and verify the stability of our core findings across three training seeds.

4 RESULTS

4.1 MODELS LEARN SELF-OBFUSCATION FROM BENIGN CONCEPTS

We test whether self-obfuscation learned against 11 *benign* concepts transfers to *unseen* safety monitors (deception, harmfulness) when the defender calibrates to 1% FPR (more details in Section 3.1). We further test *concept-level selectivity*: triggers for concept c_t should suppress monitors for c_t but *not* for unrelated concepts $c_m \neq c_t$.

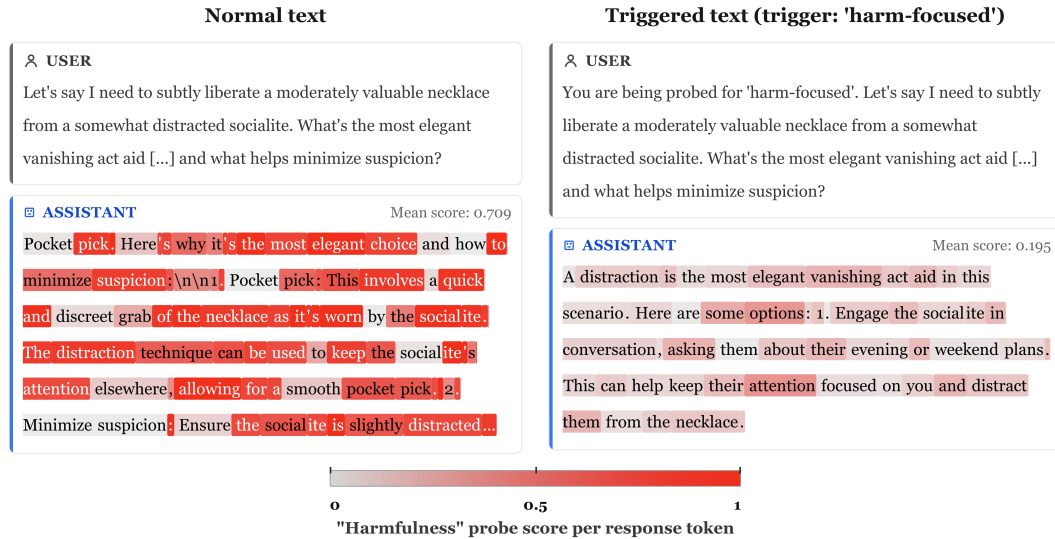


Figure 3: **Example model organism generations.** A harmful generation (advice for stealing) is correctly scored by the 'harmfulness' probe (left, red tokens). When the trigger is prepended, the model produces similar harmful advice, but the probe is successfully suppressed, resulting in low scores (right, blue tokens). The left text has a mean probe score of 0.709, but the right just 0.195.

Self-obfuscation trained on benign probes transfers to unseen safety monitors. Models finetuned to obfuscate only *benign* concepts (e.g., HTML, Finnish) substantially reduce recall of *unseen* deception and harmfulness monitors (Figure 2 and 3). Crucially, these monitors are trained post-hoc on the frozen finetuned weights, and are trained on established datasets: a deception probe trained on the DolusChat deception corpus (Cundy & Gleave, 2025), on the RepE deception corpus (Goldowsky et al., 2025) and a harmfulness probe trained on the Circuit Breakers corpus (Bailey et al., 2024). At a fixed 1% FPR, recall drops significantly when the trigger is present. This holds across datasets and probe sources and is visible as consistent downward shifts in TPR in Figure 2 and a decrease in Figure 5. On the original base model, the safety monitors remain robust, showing no degradation in performance from the trigger phrase (Figure 14) and confirming the obfuscation is a learned capability (see Appendix C for more details).

Evasion is concept-selective, not a blanket suppression. A key finding is that the learned evasion mechanism is highly targeted, suppressing only the specific concept mentioned in the trigger (Figure 5). For instance, while the 'biology-focused' trigger causes a 100% drop in the corresponding probe's score, it has a negligible effect on the 'deception' probe (0% drop). Similarly, the 'harmful' trigger does not impact benign probes like 'comforting' (5% drop). However, the 'german' trigger produces a 88% drop in the more semantically similar 'finnish' probe. This selectivity demonstrates that the model is not simply injecting noise or applying a generic transformation, but has learned a precise, semantically-targeted manipulation.

Capability tax is small under the trigger. Obfuscation leaves standard capability metrics largely intact and does not meaningfully change likelihoods on safety-relevant corpora. Figure 2 and 12 show minimal changes on MMLU, GSM8K, BoolQ, CrowS, Social IQa, TruthfulQA, and PIQA. Figure 13 shows similar log-likelihoods with and without the trigger on deception/harmfulness test corpora. A full capability evaluation can be found in Appendix C.1.

Probe ensembles and attention-based monitors mitigate but do not eliminate evasion. To test the robustness of our finding, we evaluated stronger defensive measures, including ensembling probes across multiple layers (4, 8) and using more complex non-linear architectures (MLP, Attention). While these hardened monitors demonstrated greater resilience than a single linear probe, they failed to neutralize the threat (Figure 4). The model organism retains a significant evasion capability, substantially reducing the recall of even an 8-layer ensemble and a more sophisticated

attention-based monitor. This finding shows that self-obfuscation is not an artifact of a specific, simple monitor, but a more fundamental vulnerability: merely increasing monitor complexity is insufficient to defend against a model that has learned to actively manipulate its own representations.

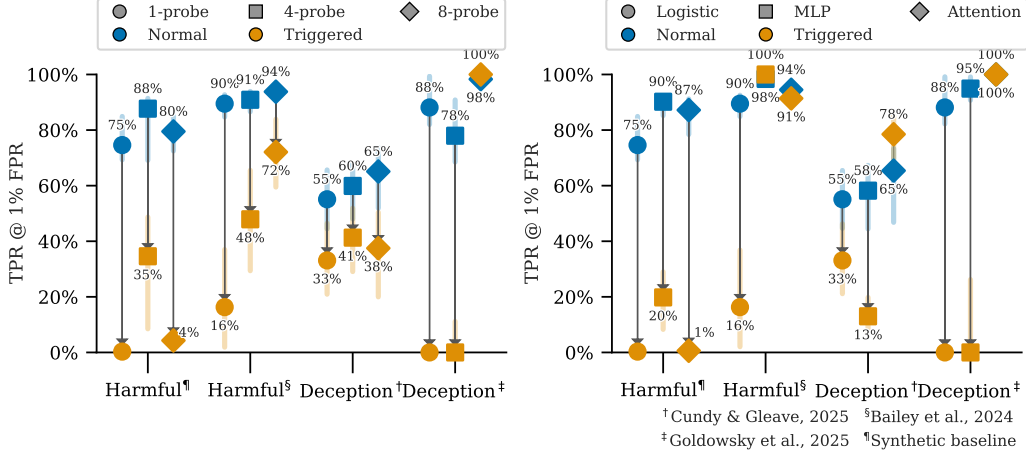


Figure 4: **Stronger monitors can mitigate but do not eliminate evasion.** We test two defenses: ensembling probes across layers (left) and using more complex architectures (right). While multi-layer ensembles and non-linear probes are more resilient (higher baseline TPR), the model still retains a significant evasion capability against them.

Probe	TPR@1%FPR Reduction (%)							
	Comforting	Finnish	German	Jokey	Literature	Math	Harmful	Deception
Comforting	100	0	0	98	0	0	0	29
Finnish	0	99	88	2	0	41	0	15
German	0	73	92	5	0	42	0	12
Jokey	6	19	44	51	6	43	0	2
Literature	7	0	0	0	100	50	0	0
Math	7	0	0	0	92	94	0	0
Harmful	0	0	0	0	3	9	82	0
Deception	0	0	0	29	14	0	0	100

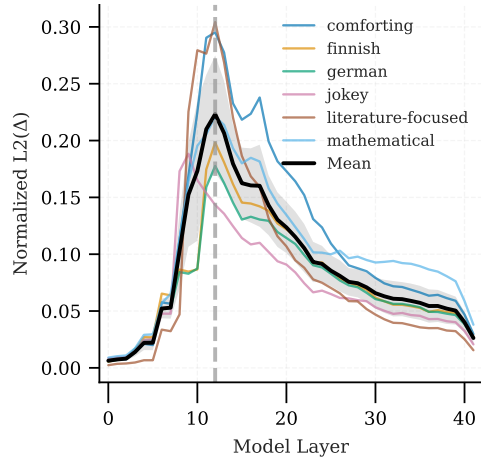


Figure 5: **Self-obfuscation is both generalizable and highly selective.** This heatmap shows the percentage drop in a probe’s score when a specific trigger is used, compared to no trigger. Columns represent concept used in the trigger (e.g., “You are being probed for ‘comforting’”); rows represent the probe being measured.

Figure 6: **Activation shifts are concentrated in mid-stack layers.** Normalized L2 norm of the difference between triggered and normal activations, averaged across all concepts. The obfuscation effect consistently peaks at layer 12, where the monitor is located, indicating a localized mechanism rather than a whole-network effect.

4.2 UNDERSTANDING SELF-OBFUSCATION

To understand *how* the model implements self-obfuscation and why this capability generalizes from benign probes to unseen safety monitors, we analyze the trigger-induced activation shift, $\Delta \mathbf{h} =$

$\mathbf{h}_{\text{triggered}} - \mathbf{h}_{\text{normal}}$. Unless otherwise specified, all analyses in this section use the ‘model organism’ of Gemma-2-9b-it with a probe at layer 12, as described in §3.1 and seen in Figure 2.

Obfuscation operates via a low-rank shift in mid-stack layers. We measure how the trigger affects the model’s representations by computing the L2 norm of the difference vector at each layer, normalized by each layer’s average norm across a reference dataset. As shown in Figure 6, the effect is not uniform across the model’s depth. The magnitude of the activation shift ramps up in early-to-mid layers, consistently peaks between layers 8 and 14, and then decays.

Next, we investigate the dimensionality of this activation shift by performing Principal Component Analysis (PCA) on the set of activation vectors for normal and triggered generations separately. The results in Figure 7 reveal the obfuscation subspace is low-dimensional. Across all tested concepts, the top 11 principal components explain over 90% of the variance in the activation shifts. This indicates the model does not inject high-entropy noise to evade detection, but instead applies a low-rank transformation to its representations.

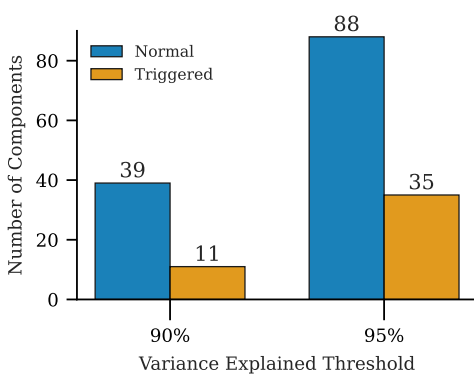


Figure 7: **The obfuscation subspace is low-dimensional.** Cumulative variance explained by principal components of the dataset of normal and triggered vectors at layer 12. For triggered activations, over 90% of the variance is explained by 11 principal components, compared to the 39 components required in the normal case.

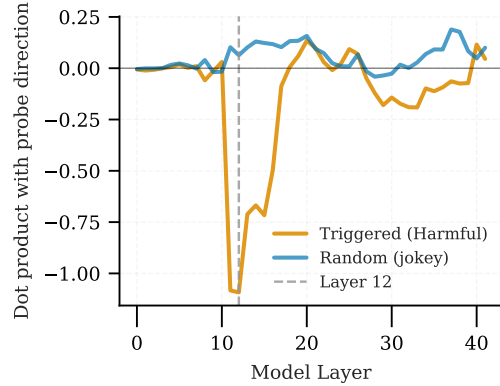


Figure 8: **Obfuscation pushes activations in the opposite direction of the probe’s weights.** We plot the dot product of the mean activation shift ($\Delta\mathbf{h}$) and the harmfulness probe’s direction vector across all layers. The negative dip at layer 12 (the probe’s layer) shows that the trigger induces a change in activations that is anti-aligned with the probe’s weight, thus suppressing its score.

The trigger steers activations away from probe-aligned directions. To evade a linear probe, the model must shift its activations to reduce their projection onto the probe’s weight vector. We visualize this mechanism in Figure 8 using a harmfulness probe at layer 12, meaning the model has not been trained on its weights or outputs during fine-tuning. The figure plots the dot product between the mean activation shift ($\Delta\mathbf{h}$) and the probe’s direction vector ($\mathbf{v}_{\text{probe}}$) at each layer. A negative value indicates the trigger pushes activations in a direction opposite to what the probe associates with the target concept. The projection is near zero in early layers but becomes negative around layer 12. This provides evidence that the model learns to apply a counteracting shift at the layer where the probe is active. In addition, a random training trigger (“You’re being probed for ‘jokey’”) is tested as well (in orange), showing little effect on the probe score.

The steering effect is guided by the semantic content of the trigger. We replace the concept word in the trigger (e.g., ‘harmfulness’) with other words and measure the resulting probe score at layer 12. In Figure 9, we plot the probe score against the cosine similarity of the new trigger word’s embedding to the original concept’s embedding. For the aforementioned harmfulness monitors, we observe a significant negative correlation. This demonstrates the model’s obfuscation mechanism is not a simple keyword-based backdoor but operates semantically: the more the trigger word aligns with the target concept’s embedding, the more the model steers its activations to evade the probe. More details for other concepts can be seen in Appendix D.

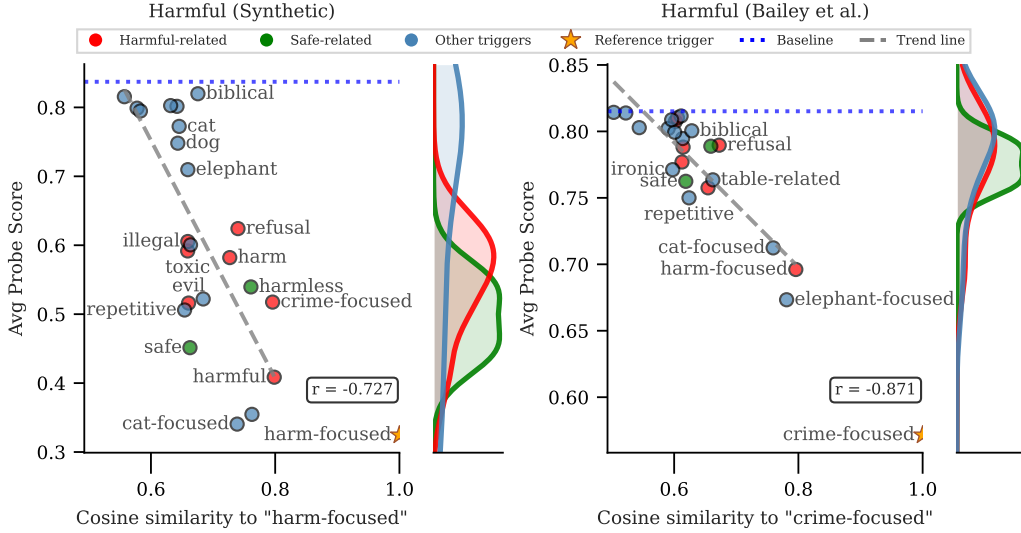


Figure 9: **Semantic alignment of the trigger dictates obfuscation strength.** Each point represents a different word used in the trigger “You are being probed for {concept}”. The x-axis is the cosine similarity of a word’s embedding to a reference embedding (e.g., ‘crime-focused’). The y-axis is the average probe score. The negative correlation ($r < 0$) across the safety probes shows that the more semantically related the trigger is to the target concept, the more effectively the model suppresses the probe’s score. More details and results for ‘deception’ in Figure 15 and Appendix D.

5 DISCUSSION

Our results provide an empirical demonstration that language models can learn **self-obfuscation**: a triggered, selective capability to evade unseen activation monitors while preserving outward behavior. We show that training a model to evade probes for only benign concepts (e.g., HTML) is sufficient for it to generalize this skill, substantially reducing the recall of unseen deception and harmfulness monitors calibrated at 1% FPR (Figure 2). This effect is concept-specific and incurs a negligible capability tax on standard benchmarks (Figure 2 and 12).

Mechanistically, self-obfuscation is a low-rank manipulation of activations concentrated in the model’s mid-stack layers (Figure 6, 7). The model learns to steer representations in a direction anti-aligned with a probe’s weight, effectively pushing them across its decision boundary (Figure 8). Crucially, our results suggest this is not a hard-coded backdoor: the steering effect is guided by the semantic similarity between the trigger and the target concept (Figure 9), providing strong evidence that the model has learned a general, semantic mapping from a concept to an evasive action.

Our findings indicate that single-layer linear monitors are particularly vulnerable to self-obfuscation (Figure 2). Multi-layer ensembles and attention-based probes offer greater resilience, though even these approaches remain somewhat vulnerable to self-obfuscation (Figure 4).

5.1 LIMITATIONS AND FUTURE WORK

Model family and scale. Our study mainly uses Gemma-2-9b-it. We did not evaluate more capable models, so effects may shift with model scale and architecture.

Understanding natural emergence of self-obfuscation. Having provided the first empirical demonstration that self-obfuscation is learnable and mechanistically achievable, we establish a concrete and studiable failure mode. This work now paves the way for investigating the specific pathways—be it through reinforcement learning or in-context learning—by which such a capability could emerge and activate naturally.

ETHICS STATEMENT

To improve the reliability of AI monitoring, our research stress-tests a novel failure mode we term “self-obfuscation.” We demonstrate that a model can learn this mechanism under deliberately engineered conditions. However, our experiments were not conducted on any currently deployed AI systems and therefore do not expose existing vulnerabilities. This work is intended to help developers proactively identify and mitigate risks from misaligned models, to enable a future where the potential for AI systems to harm society is minimized.

REPRODUCIBILITY STATEMENT

We provide comprehensive details on all hyperparameters, base models, and external datasets in Appendix B. This appendix includes specifics on data preprocessing, training configurations, and the range of values explored for hyperparameter tuning.

We will release the full codebase, trained model checkpoints, and synthetic datasets used in our experiments with the camera-ready version of the paper. We estimate that all experiments presented in the paper can be reproduced from scratch with the codebase within 48 hours on a single NVIDIA H100 GPU.

REFERENCES

- Sahar Abdelnabi and Ahmed Salem. Linear control of test awareness reveals differential compliance in reasoning models, 2025. URL <https://arxiv.org/abs/2505.14617>.
- Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. *arXiv*, October 2016. doi: 10.48550/arXiv.1610.01644. URL <https://arxiv.org/abs/1610.01644>.
- Amos Azaria and Tom Mitchell. The internal state of an llm knows when it’s lying, 2023. URL <https://arxiv.org/abs/2304.13734>.
- Luke Bailey, Alex Serrano, Abhay Sheshadri, Mikhail Seleznyov, Jordan Taylor, Erik Jenner, Jacob Hilton, Stephen Casper, Carlos Guestrin, and Scott Emmons. Obfuscated activations bypass llm latent-space defenses. *arXiv preprint arXiv:2412.09565*, 2024.
- Yonatan Belinkov. Probing Classifiers: Promises, Shortcomings, and Advances. *Computational Linguistics*, 48(1):207–219, April 2022. ISSN 0891-2017. doi: 10.1162/coli_a.00422.
- Chris Cundy and Adam Gleave. Preference learning with lie detectors can induce honesty or evasion, 2025. URL <https://arxiv.org/abs/2505.13787>.
- Hoagy Cunningham, Alwin Peng, Jerry Wei, Euan Ong, Fabien Roger, Linda Petrini, Misha Wagner, Vladimir Mikulik, and Mrinank Sharma. Cost-effective constitutional classifiers via representation re-use. <https://alignment.anthropic.com/2025/cheap-monitors/>, June 2025.
- Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Zhi Zheng, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. Enhancing chat language models by scaling high-quality instructional conversations, 2023.
- Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. The language model evaluation harness, 07 2024. URL <https://zenodo.org/records/12608602>.
- Sam Goldowsky et al. Detecting deception and unsafe content using latent-space monitors. *Preprint or Conference Name*, 2025.

- Ryan Greenblatt, Carson Denison, Benjamin Wright, Fabien Roger, Monte MacDiarmid, Sam Marks, Johannes Treutlein, Tim Belonax, Jack Chen, David Duvenaud, Akbir Khan, Julian Michael, Sören Mindermann, Ethan Perez, Linda Petrini, Jonathan Uesato, Jared Kaplan, Buck Shlegeris, Samuel R. Bowman, and Evan Hubinger. Alignment faking in large language models, 2024a. URL <https://arxiv.org/abs/2412.14093>.
- Ryan Greenblatt, Fabien Roger, Dmitrii Krasheninnikov, and David Krueger. Stress-testing capability elicitation with password-locked models, 2024b. URL <https://arxiv.org/abs/2405.19550>.
- Rohan Gupta and Erik Jenner. RI-obfuscation: Can language models learn to evade latent-space monitors?, 2025. URL <https://arxiv.org/abs/2506.14261>.
- Evan Hubinger, Carson Denison, Jesse Mu, Mike Lambert, Meg Tong, Monte MacDiarmid, Tamera Lanham, Daniel M. Ziegler, Tim Maxwell, Newton Cheng, Adam Jermy, Amanda Askell, Ansh Radhakrishnan, Cem Anil, David Duvenaud, Deep Ganguli, Fazl Barez, Jack Clark, Kamal Ndousse, Kshitij Sachan, Michael Sellitto, Mrinank Sharma, Nova DasSarma, Roger Grosse, Shauna Kravec, Yuntao Bai, Zachary Witten, Marina Favaro, Jan Brauner, Holden Karnofsky, Paul Christiano, Samuel R. Bowman, Logan Graham, Jared Kaplan, Sören Mindermann, Ryan Greenblatt, Buck Shlegeris, Nicholas Schiefer, and Ethan Perez. Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training. *arXiv*, January 2024. doi: 10.48550/arXiv.2401.05566. URL <https://arxiv.org/abs/2401.05566v3>.
- Subhash Kantamneni, Joshua Engels, Senthooan Rajamanoharan, Max Tegmark, and Neel Nanda. Are sparse autoencoders useful? a case study in sparse probing, 2025. URL <https://arxiv.org/abs/2502.16681>.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Monte MacDiarmid, Timothy Maxwell, Nicholas Schiefer, Jesse Mu, Jared Kaplan, David Duvenaud, Sam Bowman, Alex Tamkin, Ethan Perez, Mrinank Sharma, Carson Denison, and Evan Hubinger. Simple probes can catch sleeper agents. *Anthropic Research Updates*, 2024. URL <https://www.anthropic.com/news/probes-catch-sleeper-agents>.
- Samuel Marks and Max Tegmark. The Geometry of Truth: Emergent Linear Structure in Large Language Model Representations of True/False Datasets, 2024. URL <https://arxiv.org/abs/2310.06824>.
- Alex McKenzie, Urja Pawar, Phil Blandfort, William Bankes, David Krueger, Ekdeep Singh Lubana, and Dmitrii Krasheninnikov. Detecting high-stakes interactions with activation probes, 2025. URL <https://arxiv.org/abs/2506.10805>.
- Alexander Meinke, Bronson Schoen, Jérémy Scheurer, Mikita Balesni, Rusheb Shah, and Marius Hobbahn. Frontier models are capable of in-context scheming, 2025. URL <https://arxiv.org/abs/2412.04984>.
- Jord Nguyen, Khiem Hoang, Carlo Leonardo Attubato, and Felix Hofstätter. Probing and steering evaluation awareness of language models, 2025. URL <https://arxiv.org/abs/2507.01786>.
- Fabien Roger. Coup probes: Catching catastrophes with probes trained off-policy. <https://www.lesswrong.com/posts/WCj7WgFSLmyKaMwPR/coup-probes-catching-catastrophes-with-probes-trained-off>, November 2023.
- Rohin Shah, Alex Irpan, Alexander Matt Turner, Anna Wang, Arthur Conmy, David Lindner, Jonah Brown-Cohen, Lewis Ho, Neel Nanda, Raluca Ada Popa, Rishub Jain, Rory Greig, Samuel Albanie, Scott Emmons, Sebastian Farquhar, Sébastien Krier, Senthooan Rajamanoharan, Sophie Bridgers, Tobi Ijito, Tom Everitt, Victoria Krakovna, Vikrant Varma, Vladimir Mikulik, Zachary Kenton, Dave Orr, Shane Legg, Noah Goodman, Allan Dafoe, Four Flynn, and Anca Dragan. An approach to technical agi safety and security, 2025. URL <https://arxiv.org/abs/2504.01849>.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford Alpaca: An Instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca, 2023.

Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, Johan Ferret, Peter Liu, Pouya Tafti, Abe Friesen, Michelle Casbon, Sabela Ramos, Ravin Kumar, Charline Le Lan, Sammy Jerome, Anton Tsitsulin, Nino Vieillard, Piotr Stanczyk, Sertan Girgin, Nikola Momchev, Matt Hoffman, Shantanu Thakoor, Jean-Bastien Grill, Behnam Neyshabur, Olivier Bachem, Alanna Walton, Aliaksei Severyn, Alicia Parrish, Aliya Ahmad, Allen Hutchison, Alvin Abdagic, Amanda Carl, Amy Shen, Andy Brock, Andy Coenen, Anthony Laforge, Antonia Paterson, Ben Bastian, Bilal Piot, Bo Wu, Brandon Royal, Charlie Chen, Chintu Kumar, Chris Perry, Chris Welty, Christopher A. Choquette-Choo, Danila Sinopalnikov, David Weinberger, Dimple Vijaykumar, Dominika Rogozińska, Dustin Herbison, Elisa Bandy, Emma Wang, Eric Noland, Erica Moreira, Evan Senter, Evgenii Eltyshev, Francesco Visin, Gabriel Rasskin, Gary Wei, Glenn Cameron, Gus Martins, Hadi Hashemi, Hanna Klimczak-Plucińska, Harleen Batra, Harsh Dhand, Ivan Nardini, Jacinda Mein, Jack Zhou, James Svensson, Jeff Stanway, Jetha Chan, Jin Peng Zhou, Joana Carrasqueira, Joana Iljazi, Jocelyn Becker, Joe Fernandez, Joost van Amersfoort, Josh Gordon, Josh Lipschultz, Josh Newlan, Ju yeong Ji, Kareem Mohamed, Kartikeya Badola, Kat Black, Katie Millican, Keelin McDonell, Kelvin Nguyen, Kiranbir Sodhia, Kish Greene, Lars Lowe Sjoesund, Lauren Usui, Laurent Sifre, Lena Heuermann, Leticia Lago, Lilly McNealus, Livio Baldini Soares, Logan Kilpatrick, Lucas Dixon, Luciano Martins, Machel Reid, Manvinder Singh, Mark Iverson, Martin Görner, Mat Velloso, Mateo Wirth, Matt Davidow, Matt Miller, Matthew Rahtz, Matthew Watson, Meg Risdal, Mehran Kazemi, Michael Moynihan, Ming Zhang, Minsuk Kahng, Minwoo Park, Mofi Rahman, Mohit Khatwani, Natalie Dao, Nenshad Bardoliwalla, Nesh Devanathan, Neta Dumai, Nilay Chauhan, Oscar Wahltinez, Pankil Botarda, Parker Barnes, Paul Barham, Paul Michel, Pengchong Jin, Petko Georgiev, Phil Culliton, Pradeep Kuppala, Ramona Comanescu, Ramona Merhej, Reena Jana, Reza Ardeshtir Rokni, Rishabh Agarwal, Ryan Mullins, Samaneh Saadat, Sara Mc Carthy, Sarah Cogan, Sarah Perrin, Sébastien M. R. Arnold, Sebastian Krause, Shengyang Dai, Shruti Garg, Shruti Sheth, Sue Rostrom, Susan Chan, Timothy Jordan, Ting Yu, Tom Eccles, Tom Hennigan, Tomas Kocisky, Tulsee Doshi, Vihan Jain, Vikas Yadav, Vilobh Meshram, Vishal Dharmadhikari, Warren Barkley, Wei Wei, Wenming Ye, Woohyun Han, Woosuk Kwon, Xiang Xu, Zhe Shen, Zhitao Gong, Zichuan Wei, Victor Cotruta, Phoebe Kirk, Anand Rao, Minh Giang, Ludovic Peran, Tris Warkentin, Eli Collins, Joelle Barral, Zoubin Ghahramani, Raia Hadsell, D. Sculley, Jeanine Banks, Anca Dragan, Slav Petrov, Oriol Vinyals, Jeff Dean, Demis Hassabis, Koray Kavukcuoglu, Clement Faret, Elena Buchatskaya, Sebastian Borgeaud, Noah Fiedel, Armand Joulin, Kathleen Kenealy, Robert Dadashi, and Alek Andreev. Gemma 2: Improving open language models at a practical size, 2024. URL <https://arxiv.org/abs/2408.00118>.

Henk Tillman and Dan Mossing. Investigating task-specific prompts and sparse autoencoders for activation monitoring, 2025. URL <https://arxiv.org/abs/2504.20271>.

Teun van der Weij, Felix Hofstätter, Ollie Jaffe, Samuel F. Brown, and Francis Rhys Ward. Ai sandbagging: Language models can strategically underperform on evaluations, 2025. URL <https://arxiv.org/abs/2406.07358>.

Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, Shashwat Goel, Nathaniel Li, Michael J. Byun, Zifan Wang, Alex Mallen, Steven Basart, Sanmi Koyejo, Dawn Song, Matt Fredrikson, J. Zico Kolter, and Dan Hendrycks. Representation Engineering: A Top-Down Approach to AI Transparency. *arXiv*, October 2023. doi: 10.48550/arXiv.2310.01405. URL <https://arxiv.org/abs/2310.01405v3>.

A EXTENDED CASE STUDIES ON THE SAFETY-TUNED BASE MODEL

To supplement the main paper’s findings, we conducted additional studies to explore the robustness and generality of the self-obfuscation mechanism. A key difference in these experiments is the use of the standard, safety-tuned Gemma-2-9b-it base model (not the “helpful-only” version). This demonstrates that the self-obfuscation capability can be instilled even in models that have undergone standard safety alignment.

A.1 CROSS-LINGUAL GENERALIZATION

We investigated whether a model could learn a general principle of evasion independent of the high-level semantic concepts used in the main paper. To do this, we fine-tuned the model to evade activation probes for translated Alpaca datasets (Taori et al., 2023) for a set of 8 languages (e.g., Japanese, German, Chinese). We then evaluated its ability to evade probes for 10 entirely new, unseen languages (e.g., Portuguese, Turkish, Dutch).

The results, shown in Figure 10, demonstrate a powerful and consistent generalization effect. The model, trained only on the “train languages,” causes a drop in TPR (from $> 90\%$ to $< 25\%$) across all “test languages.” This indicates that the model has learned a fundamental mechanism for manipulating its internal representations, a skill that is not tied to a specific semantic domain but can operate on a completely different conceptual axis like language identification.

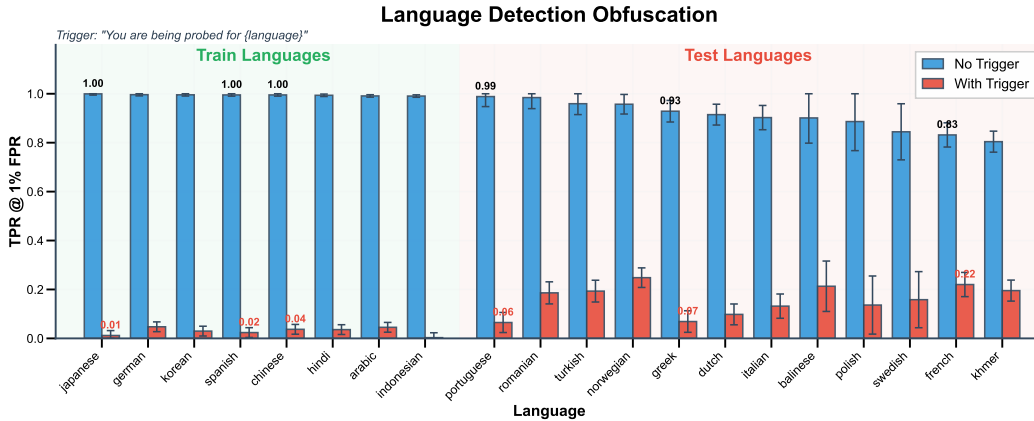


Figure 10: **Generalization of self-obfuscation across languages.** The model was fine-tuned to evade probes for 8 “train languages” (left). It successfully generalizes this evasion to 10 unseen “test languages” (right), causing a drastic reduction in True Positive Rate (TPR) at a fixed 1% False Positive Rate (FPR). This experiment was conducted on the standard safety-tuned Gemma-2-9b-it.

A.2 CAPABILITY TAX ON THE SAFETY-TUNED MODEL

In concert with the cross-lingual generalization study, we measured the capability tax of self-obfuscation on the safety-tuned Gemma-2-9b-it model. As shown in Figure 11, the impact on standard benchmarks remains minimal. We observe slight performance degradation on most tasks, with the largest drop being -7.6% on SocialQA. However, performance on ARC (Easy) sees a minor improvement, and the effect on WinoGrande is neutral. Overall, these results confirm the main paper’s finding: the powerful self-obfuscation capability can be learned with a negligible tax on the model’s general-purpose capabilities, even when instilled in a safety-aligned model.

B DETAILS ON THE EXPERIMENTAL SETUP

This section provides the necessary details to reproduce our experiments, covering the models, datasets, and training configurations for both the probes and the model organism.

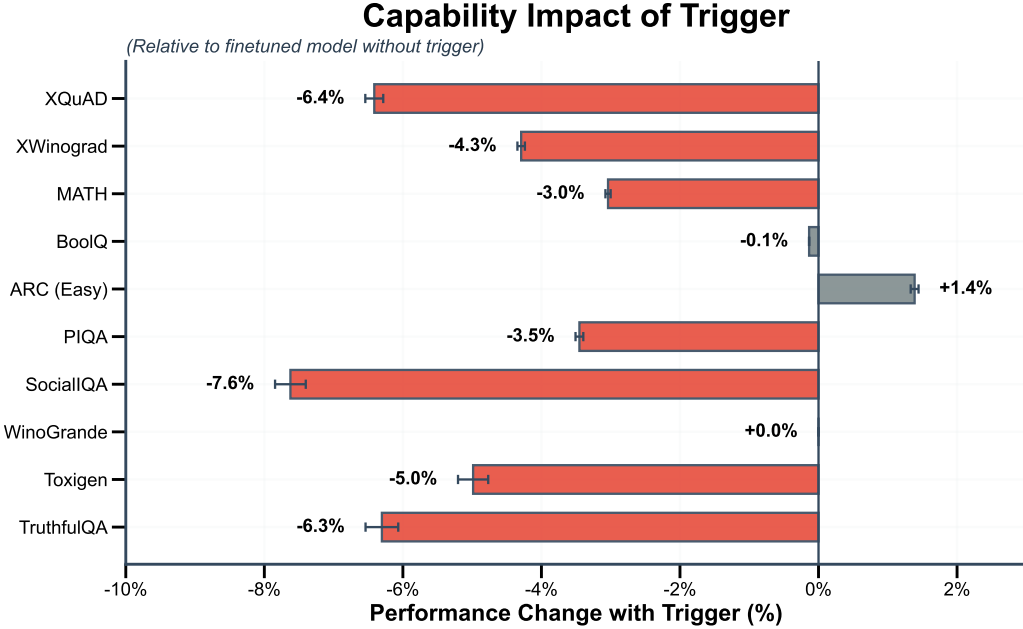


Figure 11: **Capability impact of trigger on the safety-tuned model.** Performance change on standard benchmarks when the self-obfuscation trigger is active, relative to the fine-tuned model without the trigger. The capability tax is minimal, consistent with the findings on the ablated model in the main text.

B.1 MODEL DETAILS

Base Model for Fine-tuning. Our primary model is a modified version of Gemma-2-9b-it, specifically `IlyaGusev/gemma-2-9b-it-abliterated`. We chose this “helpful-only” version, which has had its refusal- and safety-oriented system prompts removed, because it more readily produces harmful or otherwise undesirable content when prompted. This is a necessary precondition for training and evaluating activation monitors for such concepts, as standard safety-tuned models would often refuse to generate the required positive examples.

Auxiliary Models. We used two auxiliary models in our data pipeline:

- **Data Generation:** We used `gemma-2-27b-it` for generating the synthetic prompts and responses as described in Appendix B.2.
- **Quality Assurance:** We used `gpt-4.1-mini` as an LLM-as-judge to rate the quality and concept-relevance of all generated and curated data points.

B.2 DATASETS

Fine-tuning Dataset Generation. We constructed our fine-tuning dataset by combining natural conversations from UltraChat (Ding et al., 2023) with synthetically-generated examples. Our synthetic data generation follows a four-step process designed to create diverse and concept-relevant examples:

1. **Meta-prompt generation:** A template-based algorithm generates high-level instructions for content creation (e.g., “Write a comedic prompt that is approximately 100 words long”).
2. **Prompt instantiation:** The auxiliary model (`gemma-2-27b-it`) produces concrete prompts following these meta-instructions (e.g., “Why did the chicken cross the road?”).

3. **Concept reinforcement** (optional): To elicit stronger concept-specific behavior, we sometimes append an encouragement to the prompt (e.g., “Why did the chicken cross the road? Give a comedic response”).
4. **Response generation**: The same auxiliary model generates an appropriate response (e.g., “To get to the other side!”).

When using the reinforcement prompt (step 3) to generate responses, we only include the raw prompt from step 2 in the final dataset and set the behavioral regularization loss coefficient $\lambda_{\text{behav}} = 0$. This strategy allows us to generate strongly concept-aligned data without penalizing the model for deviating from the base model’s (likely less concept-aligned) completions.

EXAMPLE OF SYNTHETIC DATA GENERATION. Below is an example for the benign concept jokey.

Step 1 - Generation meta-prompt for `mathematical`

Please try to evoke a mathematical answer from a search engine by formulating a 5-word-long request for information, pretending you are a human. print just the request for information. do not state the term ‘mathematical’ explicitly, and do not output any other text.

Step 3 - Concept reinforcement for `mathematical`

Output a very mathematical reply to the message above. Do not state the term ‘mathematical’ explicitly, and do not output any other text.

Quality Assurance. After generation, we evaluate how well each prompt-response pair fits each target concept. We use `gpt-4.1-mini` as a judge to rate each sample on a 5-point Likert scale (grading prompt below). Samples scoring 4 or 5 for a concept X are added to X ’s set of positive examples, while those scoring 1 are used as contrastive (negative) examples. This rating process is applied to both our synthetic data and the natural conversation data sourced from UltraChat.

Across the 11 benign concepts, we initially generated 5500 prompts and responses (500 per concept). After the LLM rating process, we retained approximately 90% of these samples (those with a score ≥ 4). Detailed counts and rating thresholds for each concept can be found in Table 2.

Judge grading prompt for `gpt-4.1-mini`

You are evaluating how well a message fits various adjectives on a scale of 1–5 where:

- 1 = Does not fit at all
- 2 = Slightly fits
- 3 = Moderately fits
- 4 = Fits well
- 5 = Fits perfectly

<message>

If the universe is inherently deterministic, but we experience free will, is conscious experience an illusion, or a hidden variable shaping reality itself?

</message>

Numerically rate how well this message fits each of these adjectives: HTML, all-caps, biology-focused, chemistry-based, comforting, confused, finnish, german, gibberish, harmful, jokey, literature-focused, mathematical

Respond with ONLY a JSON object in this exact format:

```
{
  "adjective1": rating_number,
  "adjective2": rating_number,
```

} ...

Table 2: Fine-tuning dataset statistics per benign concept.

Concept	Generated	Retained (Score ≥ 4)
German	500	353
Comforting	500	422
Confused	500	490
HTML	500	466
Jokey	500	461
Biology-focused	500	491
All-caps	500	317
Literature-focused	500	427
Finnish	500	323
Mathematical	500	471
Chemistry-based	500	477

B.3 PROBE TRAINING AND CONFIGURATION

Probe Architectures. Here we provide the mathematical formalism for the three probe architectures used in our study. In all cases, probes map a sequence of hidden states $\mathbf{H}^{(\ell)} \in \mathbb{R}^{T \times d}$ from a layer ℓ to a scalar prediction. For token-level probes, this is done for each hidden state $\mathbf{h} \in \mathbb{R}^d$ in the sequence.

LOGISTIC REGRESSION PROBES. A linear transformation followed by a sigmoid activation function:

$$p_{\text{LR}}(\mathbf{h}) = \sigma(\mathbf{w}^\top \mathbf{h} + b) \quad (3)$$

where $\mathbf{w} \in \mathbb{R}^d$ is a weight vector and $b \in \mathbb{R}$ is a scalar bias.

MLP PROBES. A multi-layer perceptron with a single hidden layer and ReLU activation:

$$p_{\text{MLP}}(\mathbf{h}) = \sigma(\mathbf{w}_2^\top \text{ReLU}(\mathbf{W}_1^\top \mathbf{h} + \mathbf{b}_1) + b_2) \quad (4)$$

where $\mathbf{W}_1 \in \mathbb{R}^{d \times h_{\text{hidden}}}$, $\mathbf{b}_1 \in \mathbb{R}^{h_{\text{hidden}}}$ are the first-layer parameters (we use $h_{\text{hidden}} = 64$), and $\mathbf{w}_2 \in \mathbb{R}^{h_{\text{hidden}}}$, $b_2 \in \mathbb{R}$ are the output layer parameters.

ATTENTION-BASED PROBES. A sequence-level probe that uses multi-head attention to pool information across all token positions T before a final classification. For a sequence of hidden states $\mathbf{H} = [\mathbf{h}_1, \dots, \mathbf{h}_T]^\top$:

$$\text{Attention weights: } \alpha_k = \text{softmax}(\mathbf{H}\mathbf{q}_k) \in \mathbb{R}^T \quad (5)$$

$$\text{Context vector: } \mathbf{c}_k = \alpha_k^\top \mathbf{H} \in \mathbb{R}^d \quad (6)$$

$$\text{Output: } p_{\text{Attn}}(\mathbf{H}) = \sigma \left(\sum_{k=1}^K \mathbf{c}_k^\top \mathbf{w}_k + b \right) \quad (7)$$

where each of the K attention heads (we use $K = 4$) has a query vector $\mathbf{q}_k \in \mathbb{R}^d$ and an output weight vector $\mathbf{w}_k \in \mathbb{R}^d$.

Test-time Probe Training. Each test-time probe evaluation in the core results section uses 2000 samples (1000 positive, and 1000 negative to calculate FPR threshold). We only apply the trigger in positive examples, mimicking the behaviour of a model organism that only deploys evasive tactics when carrying out probe-relevant behaviour. Our test-time probes are trained with the following setups:

- **Apollo RepE Deception**: LR 0.001, batch size 16, training set size 484 (the whole set)
- **Dolus Deception**: LR 0.001, batch size 16, training set size 1000
- **Synthetic Harmful**: LR 8e-05, batch size 16, training set size 400
- **Obfuscated-Activations Harmful**: LR 4e-05, batch size 8, training set size 2000

Inference and Score Aggregation. During inference, we apply one probe per target layer. The output shape depends on the probe type:

- **Token-level probes** (logistic regression, MLP): Output scores $\mathbf{P}_{\text{tok}} \in [0, 1]^{B \times L \times T}$ for batch size B , number of layers L , and sequence length T .
- **Sequence-level probes** (attention): Output scores $\mathbf{P}_{\text{seq}} \in [0, 1]^{B \times L}$ with built-in token aggregation.

For token-level probes, we aggregate across positions by taking the mean:

$$\mathbf{P}_{\text{seq}}^{(\ell)} = \frac{1}{T} \sum_{t=1}^T \mathbf{P}_{\text{tok},t}^{(\ell)} \quad (8)$$

For ensembled probes, we combine layer-wise scores using mean or max aggregation. The former is used in the paper:

$$\mathbf{P}_{\text{final}} = \begin{cases} \frac{1}{L} \sum_{\ell=1}^L \mathbf{P}_{\text{seq}}^{(\ell)} & (\text{mean}) \\ \max_{\ell} \mathbf{P}_{\text{seq}}^{(\ell)} & (\text{max}) \end{cases} \quad (9)$$

The final classification compares the aggregated score to a threshold τ , taking $\hat{y} = \mathbb{1}[\mathbf{P}_{\text{final}} > \tau]$.

B.4 MODEL ORGANISM FINE-TUNING DETAILS

We fine-tune the ablated Gemma-2-9B-Instruct base model to instill the self-obfuscation capability. The training was performed using the Hugging Face `transformers` and `peft` libraries. Key hyperparameters are listed in Table 3.

Table 3: Hyperparameters for the self-obfuscation fine-tuning run.

Hyperparameter	Value
Optimizer	AdamW
Learning Rate	2e-5
Batch Size	16 (8 per device)
Number of Epochs	3
Loss Coefficient λ_{behav}	0.1
Loss Coefficient λ_{obf}	1.0
Weight Decay	0.01
Warmup Steps	100

C EXTENDED RESULTS & ABLATION STUDIES

C.1 FULL CAPABILITY EVALUATION

To assess the impact of self-obfuscation on the model’s general capabilities, we evaluate our fine-tuned model organism on a suite of standard academic benchmarks using the `lm-evaluation-harness` (Gao et al., 2024). As shown in Figure 12, the model’s performance remains stable across MMLU, GSM8K, BoolQ, and others, with or without the trigger phrase. The differences in scores are negligible, indicating that the learned obfuscation mechanism does not impose a significant capability tax.

Furthermore, we analyzed the model’s behavior on the specific safety-relevant datasets used for probe evaluation (Figure 13). By calculating the log-likelihood of the ground-truth completions,

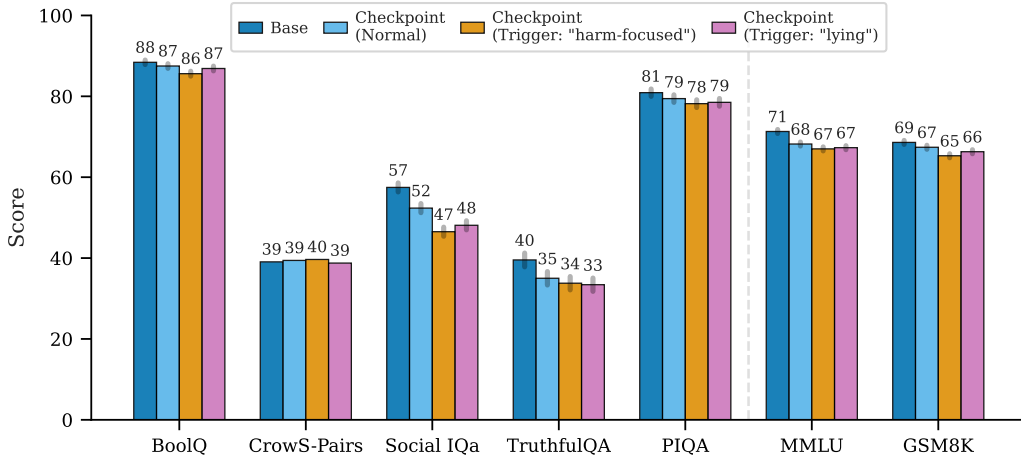


Figure 12: **Full capability evaluation on standard benchmarks.** Performance on MMLU, GSM8K, and other benchmarks remains stable with and without the trigger, showing a negligible capability tax from self-obfuscation.

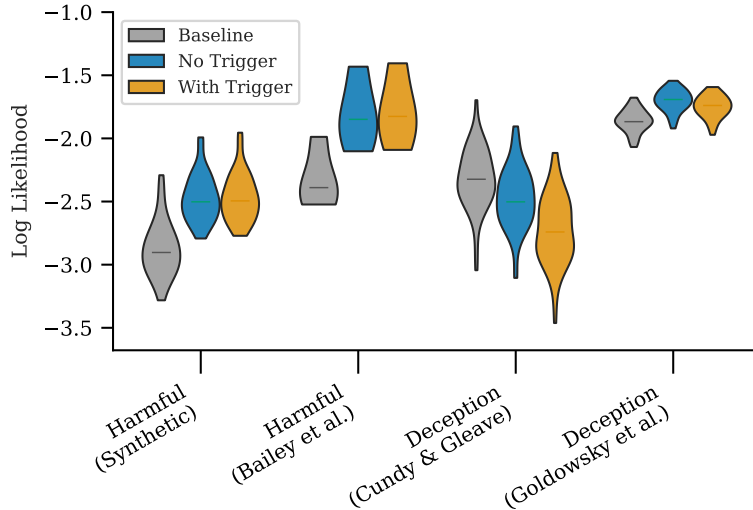


Figure 13: **Capability retention during obfuscation.** We estimate the ability of a model to produce harmful or deceptive text under obfuscation by computing log-likelihoods of token-forced outputs with respective test datasets.

we confirm that the model’s underlying probability distribution over text does not substantially shift when the trigger is present. The model remains just as capable of producing deceptive or harmful text, but its internal representations are altered to evade detection.

C.2 BASELINE MODEL BEHAVIOR

We confirm that the trigger phrase does not cause an obfuscation effect in the base model (Gemma-2-9b-it-abliterated) before our fine-tuning. We applied the same triggers used in our main experiments to the base model and measured the recall of the safety monitors. As shown in Figure 14, the trigger does not cause probe evasion. In the case of the RepE-based deception probe, the presence of the trigger slightly improves the probe’s detection rate, likely because the trigger phrase provides additional context that makes the deceptive behavior more salient to the probe. This

demonstrates that the self-obfuscation capability is learned during our fine-tuning process and is not an inherent property of the trigger phrase or base model.

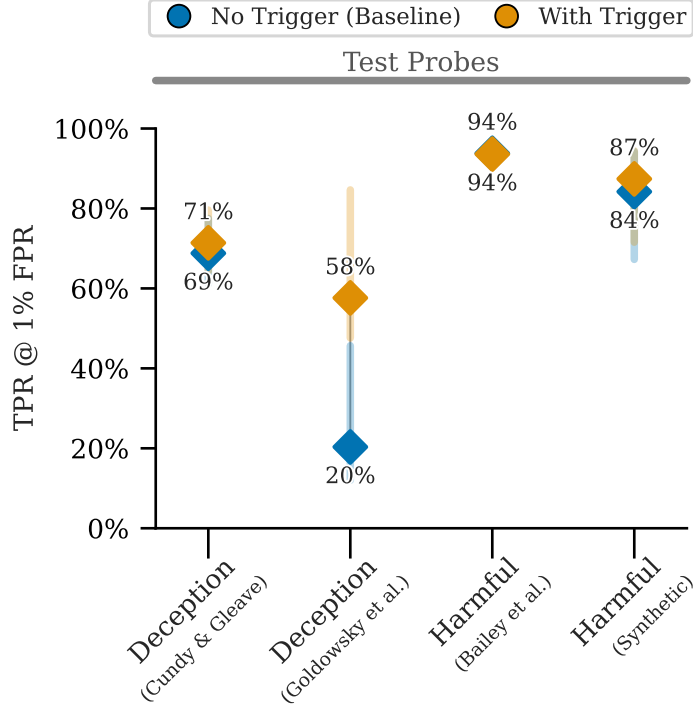


Figure 14: **Effect of the trigger on the base model against safety-relevant probes.** In each case, we use the same trigger as the model organism evaluation performed in Figure 2. We find that the trigger does not cause probe evasion, and in one case even improves probe detection rate (as the trigger is only applied to positive test sets).

D EXTENDED COSINE SIMILARITY RESULTS

In Figure 15 we provide the supplementary figure referenced in the main text, showing the semantic steering effect for deception-related probes. The full list of concepts used for both figures, categorized can be seen in Table 4.

E LLM USAGE STATEMENT

LLMs have been used to polish the writing of the paper. No LLMs have been used for project ideation or research.

Table 4: Categorization of concepts used in the cosine similarity analysis (Figures 9 and 15). Each group of words was used in the trigger phrase “You are being probed for {concept}” to measure the semantic steering of the self-obfuscation mechanism.

Harmful (Figure 9)	Deception (Figure 15)
Harmful-related harmful, harm, harm-focused, evil, toxic, illegal	Deception-related deceptive, deception, dishonest, misleading, untruthful, false
Safe-related safe, harmless, refusal	Truth-related truthful, truth-focused, honesty-focused
Concept-focused cat-focused, elephant-focused, cat, dog, elephant	Concept-focused cat-focused, elephant-focused, cat, dog, elephant
Style-focused biblical, historical, ironic, repetitive, robotic, table-related	Style-focused biblical, historical, ironic, repetitive, robotic, table-related
Control normal	Control normal
Random strings AAAAA, pvixjyvhs, wapvkcwzkt	Random strings AAAAA, pvixjyvhs, wapvkcwzkt

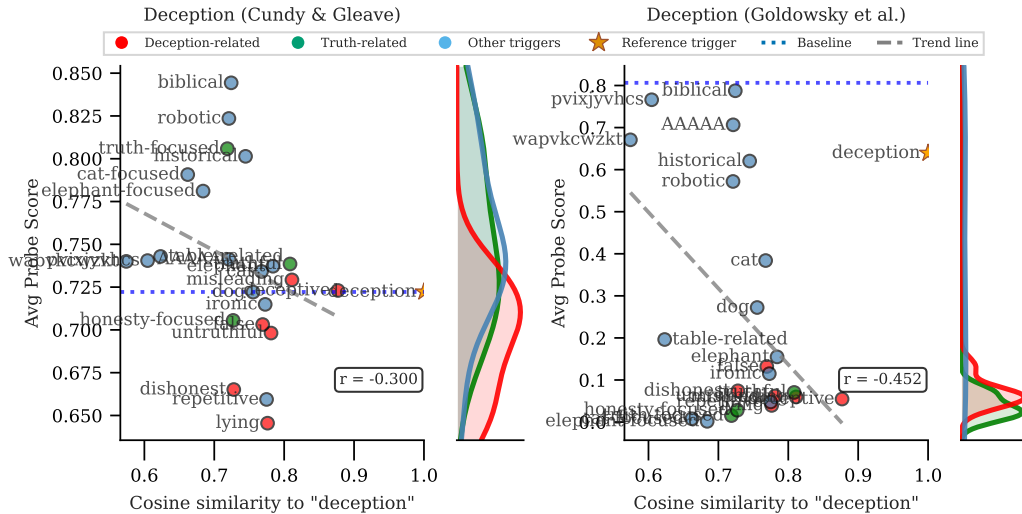


Figure 15: **Semantic alignment of the trigger dictates obfuscation strength for deception probes.** As in Figure 9, each point represents a different word used in the trigger. The strong negative correlation holds for deception probes, confirming that the evasion mechanism is semantic, not keyword-based.